Jorge Palomino y Raúl Matellano

PRÁCTICA 1

Pregunta 1.1: ¿Cómo evoluciona el número de nodos al aumentar la profundidad?
Al aumentar la profundidad el número total de nodos aumenta de manera exponencial.

Pregunta 1.2: ¿Es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida? Justifica tu respuesta.
Como hemos dicho en la pregunta anterior, al ser el crecimiento exponencial, no es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida. Una partida puede alcanzar los 60 turnos y el número de estados posibles asciende a cifras muy grandes que son imposible de procesar por cualquier ordenador actual en un tiempo razonable.

Pregunta 1.3: ¿Qué diferencias observáis entre BP y BA al explorar árboles de juego?
La búsqueda en profundidad baja verticalmente por las ramas del árbol; la búsqueda en anchura explora horizontalmente nivel por nivel. Esto conlleva a que la búsqueda en profundidad priorice caminos largos y profundos sin asegurar la ruta más corta, por lo que la búsqueda en anchura resulta más adecuada para encontrar la solución con el menor número de movimientos posibles.

Pregunta 1.4: ¿Cuál de los dos algoritmos requiere más memoria? Explica por qué.
La búsqueda en anchura requiere más memoria porque guarda en la cola todos los estados correspondientes a una misma altura previa al paso hacia el siguiente nivel, por lo que, debido al crecimiento exponencial del cual hemos hablado antes, el número de nodos guardados crece exageradamente.

Pregunta 1.5: Suponed que cada hoja contiene una puntuación fija (por ejemplo, en función del número de fichas negras). ¿Permiten estos algoritmos determinar cuál es la mejor jugada para un jugador racional? Justificad vuestra respuesta.

No, estos algoritmos no sirven por sí solos para elegir la mejor jugada frente a un adversario racional. La búsqueda en profundidad y la búsqueda en anchura solo buscan llegar a una meta predeterminada, como puede ser capturar una esquina, pero no contemplan que el rival juegue para minimizar nuestras opciones. Para eso se necesitan algoritmos basados en juegos y adversarios, como Minimax.

Pregunta 1.6: Define 3 funciones heurísticas con varios niveles de complejidad (puedes empezar a partir de la heurística entregada simpleHeuristic), implementando las funciones ya definidas heuristic1, heuristic2, heuristic3 (de menor a mayor complejidad): ¿qué características parecen ayudar a resolver el problema de capturar una esquina en un menor número de pasos?

Heurística 1: Se basa únicamente en la ventaja material que calcula la diferencia entre el número de fichas propias y las del rival.
Heurística 2: Combina la ventaja material con el control de las esquinas del tablero, dándole un peso mucho mayor a el tener estas casillas clave.
Heurística 3: Añade a la heurística 2 un factor de movilidad, restando los movimientos legales disponibles para el rival para limitar sus opciones.

Las características que realmente marcan la diferencia son la priorización posicional y el control espacial que en nuestro caso lo hacen las heurísticas 2 y 3, ya que guían a A* mucho más rápido hacia las esquinas y reducen el número de nodos explorados.

Pregunta 1.7: Analiza qué ocurre con los métodos de búsqueda y las distintas heurísticas propuestas si se intenta resolver el problema de capturar todas las esquinas, es decir, en el main, instancia el problema AllCornersReversiSearchProblem en lugar de CornerReversiSearchProblem.

Para corroborar la limitación de los algoritmos, realizamos la prueba tanto en un tablero reducido de 6x6 como en el de 8x8. Mientras que en el tablero 6x6 DFS localizó el objetivo a profundidad 27, en el de 8x8 la profundidad del primer estado objetivo encontrado se disparó a 52 movimientos. En ambos casos, el programa colapsó al intentar ejecutar BFS (tuvimos que realizar CTL+C). Al aumentar a 8x8 el tablero, no solo aumenta el factor de ramificación medio, sino que la profundidad a la que se encuentra la meta (4 esquinas) casi se duplica. Esto demuestra que, sin una función heurística admisible que estime la distancia real a las esquinas y pode las ramas inútiles en A, buscar objetivos tan lejanos en juegos como Reversi es inabordable

Pregunta 1.8: En base a tus resultados, ¿puede la función heurística equivocarse? ¿qué aspectos habría que tener en cuenta para hacer que A* sea óptimo?

La función heurística si puede equivocarse. El error ocurre cuando la heurística estima mal los movimientos restantes hasta el objetivo. En nuestro caso, las heurísticas fallan porque evalúan la ventaja en la partida y no la distancia real hacia las 4 esquinas.

Para garantizar matemáticamente que A* sea óptimo, la heurística debe cumplir dos propiedades: Admisibilidad, que nunca debe sobreestimar el coste real para alcanzar el objetivo ($h(n) \leq h^*(n)$); y por otro lado monotonía, que debe cumplir $h(n) \leq c(n, a, n') + h(n')$. Esto asegura que el algoritmo sea eficiente sobre grafos, ya que garantiza que la primera vez que A* expande un nodo, ha llegado a él por el camino más corto posible.

2. Minimax con Alfa-Beta

El algoritmo Minimax con poda Alfa-Beta es una optimización del Minimax tradicional pensada para reducir la cantidad de nodos que evaluamos en el árbol de juego. La principal ventaja de esta poda es que nos permite acelerar el tiempo de manera considerable sin cambiar la decisión final respecto al algoritmo sin optimizar. Para aplicar la poda, mantenemos y actualizamos dos valores clave a lo largo de la exploración recursiva: alfa, que guarda la mejor puntuación que el jugador MAX tiene garantizada hasta ese instante, y beta, que representa la mejor alternativa asegurada para MIN. Conforme bajamos por el árbol, comparamos estos límites en cada nodo. Si en un nodo de tipo MIN el valor estimado es menor o igual que Alfa, se interrumpe la búsqueda en esa rama, ya que sabemos que MAX nunca elegirá una ruta con un resultado inferior al que ya tenía asegurado. De forma simétrica, en los nodos de MAX se descarta la rama en cuanto el valor supera a Beta, pues MIN evitaría esa opción.

Para implementar esta técnica hemos modificado la clase MinimaxAlphaBetastrategy en strategy.py, descomponiendo la lógica en dos métodos auxiliares. max_value simula nuestro turno, buscando maximizar la utilidad y aplicando la poda beta y min_value simula el turno del rival, buscando minimizar la puntuación y aplicando la poda alfa. En el método next_move, la búsqueda arranca desde el nodo raíz, devolviendo tanto el valor estimado como el mejor estado sucesor. Además, en los nodos hoja controlamos el tiempo invertido por la heurística para evitar superar el límite de tiempo.

Para ver esto con un caso práctico, supongamos que desde el nodo raíz el jugador MAX tiene dos opciones principales, las ramas A y B. Al explorar la rama A, vemos que la mejor opción que MIN le dejará conseguir a MAX vale 5, por lo que el nodo A toma el valor 5 y el parámetro Alfa se actualiza a ese mismo valor.
Posteriormente, al empezar a evaluar la rama B, analizamos la primera hoja y obtenemos un 3. En ese punto, MIN ya sabe que la rama B devolverá como mucho un 3. Como MAX ya cuenta con la rama A que le asegura un 5, podemos descartar B sin necesidad de revisar el resto de sus hijos. Aquí es donde entra en juego la poda, evitando evaluar el resto de ramas y reduciendo el coste de procesamiento.

Para verificar los 4 apartados del enunciado hemos resuelto a mano el árbol de prueba y luego lo hemos comparado con la salida que genera demo_simple_game_tree.py con verbose = 3.
Minimax puro: tras evaluar las ramas B(3), C(2) y D(5), la raíz A selecciona el valor óptimo de Max con la jugada AD. La salida confirma estos valores en el primer turno.
Poda Alfa-Beta: la secuencia de intervalos obtenida con verbose = 3 coincide con los cálculos manuales y refleja las dos podas que se producen.
A: [-inf, inf] → B: [-inf, inf] → B: [-inf, 4] → A: [3, inf] → C: [3, inf] → G: [3, inf] → G: [3, inf] → A: [3, inf] → D: [3, inf] → D: [3, 5] → J: [3, 5] → J: [4, 5] → J: [4, 5]

El valor final en la raíz es 5 y la secuencia de movimientos hasta el final de la partida es AD seguido de DI.

Para comprobar el rendimiento de la poda en un problema real, medimos los tiempos de ejecución en el tablero inicial de 8x8 utilizando la librería timeit. Hicimos la comparativa a profundidades 3 y 4 utilizando tanto la heurística determinista del profesor como nuestra heurística propia. Los resultados son los siguientes

Heurística	Profundidad	T Minimax	T Alfa-Beta	Disminución
Heuristic1	3	0.0139	0.0078	1.79x
Heuristic1	4	0.0660	0.0246	2.68x
Luna1	3	0.0135	0.0086	1.57x
Luna1	4	0.0662	0.0305	2.17x
JavisFather	3	0.0143	0.0079	1.82x
JavisFather	4	0.0685	0.0332	2.06x
UltimaEsperanza	3	0.0143	0.0077	1.85x
UltimaEsperanza	4	0.0693	0.0322	2.15x

3. Documentación del diseño de la heurística

a) Revisión de trabajos previos sobre estrategias para el juego Reversi:

Antes de ponernos a programar las heurísticas del torneo estuvimos leyendo varios artículos sobre qué factores deciden de verdad quién gana en el Reversi.
Vimos un artículo de Sannidhanam y Muthukaruppan donde mencionaba que las fichas fronterizas son muy peligrosas a mitad de partida porque te las pueden dar la vuelta muy fácilmente. Otro concepto clave que sacamos de Van der Ree fue la movilidad. La idea es tener siempre más opciones de movimiento que el rival para ir

arrinconando y ahogarle. También tuvimos en cuenta la estabilidad descrita por Rosenbloom, buscando posiciones que no te pueden quitar en toda la partida, como las esquinas. Por último, nos basamos en Buro para la paridad de turno, que demuestra una ventaja enorme de ser quien pone la última ficha al acabar el torneo.

b) Descripción del proceso de diseño

El proceso de diseño de nuestras heurísticas se ha planteado de forma evolutiva a lo largo de los tres torneos de la asignatura. Para comparar su efectividad las probamos en enfrentamientos internos frente a las tres referencias del profesor (dummy, random y heuristic)

- **Torneo 1:** Decidimos enviar tres estrategias con el objetivo de evaluar qué conceptos tácticos funcionaban mejor. Dos de ellas sirvieron como control: La primera llamada “Solution2” era una aproximación minimalista que únicamente evaluaba la diferencia de esquinas capturadas, mientras que “Solution3 “le sumaba un componente codicioso basado en maximizar la diferencia de fichas totales. Sin embargo, nuestra estrategia principal y más compleja fue " Luna1". En esta heurística diseñamos una matriz de pesos posicionales dinámicos. Dividimos el tablero en cuatro zonas clave (esquinas, bordes, "casibordes" y centro). En lugar de asignarles puntuaciones fijas, hicimos que los pesos evolucionarán en función de la partida. En cuanto a los resultados frente a dummy random y heuristic fueron los siguientes: Solution3: 100% de victorias vs Dummy (4/4), 75% vs Random (3/4), y 75% vs Heuristic (3/4). Luna1: 100% de victorias vs Dummy (4/4), 50% vs Random (2/4), y 75% vs Heuristic (3/4).
- **Torneo 2:** Tras el primer torneo obtuvimos los siguientes resultados en la clasificación de todas las heurísticas: Luna = 65/121 y Solution3= 89/121. Con el objetivo de superar los resultados del primer torneo, decidimos rediseñar por completo nuestra estrategia creando la versión "JavisFather" . Descartamos el conteo de fichas y la matriz estática, sustituyéndolos por un análisis dinámico de fronteras y un control estricto de la paridad matemática para asegurar el último movimiento de la partida. Al someter esta nueva versión a nuestro banco de pruebas, el rediseño demostró ser un éxito rotundo, logrando un pleno absoluto en las pruebas locales. “JavisFather”: 100% de victorias vs Dummy (4/4), 100% vs Random (4/4), y 100% vs Heuristic (4/4).
- **Torneo 3**: A pesar de lograr un 100% de victorias en los test locales frente a las referencias, los resultados oficiales del Torneo 2 situaron a "JavisFather" en el puesto 54 de 129. Tras estos resultados realizamos esta última heurística llamada “UltimaEsperanza” que también consiguió un 100% de victorias frente a las pruebas locales.

C)

Nuestra heurística definitiva, "UltimaEsperanza", es una evolución de nuestra versión anterior, “JavisFather”. “JavisFather” otorgaba un valor de 100 ponderado a las esquinas, penaliza de forma estática las casillas adyacentes de peligro si la esquina estaba libre, premiaba los bordes seguros y restaba puntos si nuestras fichas estaban en contacto con espacios vacíos.

A pesar de todo buscamos alguna mejora a “JavisFather”, las cuales fueron excelentes, ya que tras hacer 10 partidas entre estas dos, esta última ganó 10/10:

Primero, eliminamos los cortes abruptos basados en condiciones estrictas de casillas vacías, sustituyéndolos por la función ((25 - vacías) / 4) para que el peso de las fichas evolucione suavemente de penalizar la acumulación inicial a premiarla al final. Además, cambiamos el valor a las casillas adyacentes a las esquinas . Antes se castigaban siempre por igual; ahora su valor cambia según la situación del tablero: si la esquina correspondiente sigue libre, se penalizan (-300 y -100) para evitar regalarle una ventaja al rival. En cambio, si esa esquina ya es nuestra, esas mismas casillas adyacentes pasan a premiarse (+50 y +20), lo que ayuda a la IA a rodearla y construir un bloque seguro e inamovible de fichas estables. Por último, elevamos el valor absoluto de las esquinas a 1000 puntos, aumentamos el peso defensivo de la frontera de 5 a 15 y reparamos la lógica de lectura evaluando estrictamente None, lo que garantiza una gestión segura frente a las casillas bloqueadas con obstáculos del tablero.